

PYTHON SELENIUM AUTOMATION

PROJECT REPORT

Submitted by

Srijita Ghosh

Section: A

Roll No.: 42

Enrollment No.: 12023002028043

B.Tech

Computer Science & Engineering (Artificial Intelligence & Machine Learning)

Topic: Python Selenium Automation

CERTIFICATE

This is to certify that the project report entitled “Python Selenium Automation” has been prepared and submitted by Srijita Ghosh, Section A, Roll No. 42, Enrollment No. 12023002028043, as part of the academic requirements of the B.Tech programme in Computer Science & Engineering (Artificial Intelligence & Machine Learning).

The report presents the study and implementation of web automation and automated testing concepts using Python and Selenium, based on the project materials supplied for this work.

Faculty/Guide Name: _____

Department: _____

Institution: _____

Date: _____

Signature: _____

DECLARATION

I, Srijita Ghosh, Section A, Roll No. 42, Enrollment No. 12023002028043, declare that this project report entitled “Python Selenium Automation” has been prepared for academic purposes and represents my study and work based on the project materials provided for the assignment.

I understand that the report should be submitted in accordance with the academic requirements of the B.Tech CSE (AI & ML) programme. Project-specific facts and implementation details in this report are presented from the supplied materials; information not specified in those materials is not represented as an observed project result.

Student Signature: _____

Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty members and mentor for their guidance and support during the preparation of this project report. I also acknowledge the contribution of my department and institution in providing the academic environment in which the project could be studied and documented.

I am thankful for the learning resources, technical documentation, development tools, and project materials that supported my understanding of Python, Selenium WebDriver, web elements, locators, automated testing, PyTest, and framework organization. I also appreciate everyone who provided feedback and encouragement during the preparation of the project.

TABLE OF CONTENTS

1.	1. Abstract
2.	2. Introduction
3.	3. Selenium and Web Automation
4.	4. Application Under Test
5.	5. Problem Statement
6.	6. Objectives
7.	7. Scope of the Project
8.	8. Technologies and Tools Used
9.	9. System Requirements
10.	10. Selenium Architecture
11.	11. Project Framework Architecture
12.	12. Page Object Model
13.	13. Configuration Management
14.	14. CSV Test Data
15.	15. PyTest Configuration and Test Discovery
16.	16. Selenium Locators
17.	17. Selenium WebDriver Commands
18.	18. Explicit and Implicit Waits
19.	19. Assertions
20.	20. Browser Test
21.	21. Login Test
22.	22. Product Search Test
23.	23. Automatic Failure Screenshots
24.	24. Logging
25.	25. Test Cases
26.	26. Implementation
27.	27. Code Explanation
28.	28. Test Execution
29.	29. Results and Analysis
30.	30. Test Report and HTML Reporting
31.	31. Screenshots
32.	32. Advantages
33.	33. Limitations
34.	34. Challenges Faced
35.	35. Future Scope
36.	36. Conclusion
37.	37. References
38.	38. Viva Questions and Answers
39.	Appendix A. Key Project Files

1. Abstract

Selenium is a browser-automation technology used to control web browsers programmatically and is widely used for functional and regression testing of web applications. Python provides a concise programming environment for writing Selenium automation scripts, while PyTest provides test discovery, execution, assertions, and reporting support.

The supplied project is a Selenium E-Commerce Automation Framework developed as a capstone assignment for the TutorialsNinja Demo e-commerce website. The framework uses Python, Selenium WebDriver, PyTest, Page Object Model (POM), CSV-based login test data, configuration management, logging, explicit/implicit wait concepts, automatic screenshots on failure, and HTML test reporting. The implemented test modules cover opening the website, valid login, and product search.

The course material additionally emphasizes identifying web elements and their attributes, using Selenium locators such as ID, Name, Class Name, Tag Name and Link Text, working with multiple elements, CSS selectors and wildcard selectors, and locating nested elements through CSS child selectors. These concepts provide the foundation for reliable element identification in Selenium automation. □ filecite□turn2file1□L2-L12□

The report documents the architecture, setup, framework organization, implementation, test cases, reporting mechanisms, limitations, and future enhancements. Where exact execution metrics are not available in the supplied material, the report explicitly states that they are not specified rather than inventing values.

2. Introduction

2.1 Web Automation

Web automation is the use of software to perform browser-based actions that would otherwise be performed manually. Typical actions include opening a URL, locating controls, entering values, clicking buttons, reading page content, validating expected outcomes, and closing the browser.

Manual testing remains useful for exploratory and usability-oriented work, but repetitive functional checks can consume significant time. Automation makes repeatable checks easier to execute consistently and supports regression testing after application changes.

2.2 Manual Testing vs Automation

Aspect	Manual Testing	Automation Testing
Execution	Performed by a tester	Performed by an automation script
Repeatability	Requires repeated manual effort	Can be rerun consistently
Speed	Depends on human interaction	Usually faster for repetitive checks
Regression	Can become time-consuming	Suitable for repeat regression suites
Consistency	May vary with manual execution	Same scripted steps are repeated
Reporting	Often manually documented	Can be generated automatically

2.3 Selenium

Selenium is an open-source ecosystem for automating web browsers. Selenium WebDriver provides an interface through which an automation program sends browser commands and interacts with web pages.

2.4 Python with Selenium

Python is useful for Selenium because it has readable syntax, a large ecosystem, and straightforward support for organizing reusable test classes, utility modules, data readers, and assertions.

2.5 Importance of Automated Testing

- Speed for repetitive test execution.
- Repeatability of defined test scenarios.
- Consistent validation through assertions.
- Support for regression testing.
- Reduced manual effort for routine browser checks.
- A reusable foundation for expanding the test suite.

3. Selenium and Web Automation

3.1 Selenium WebDriver

WebDriver is the primary Selenium interface used by the project to create a browser session and perform browser operations. The Python test code interacts with the WebDriver object, which in turn controls the browser.

3.2 Web Elements

A web element is an interactive or informational object in a page's HTML DOM, such as a text field, link, button, heading, menu item, or product result. Reliable automation depends on identifying these elements using appropriate locators.

3.3 Locator Selection

The supplied course outcome specifically covers ID, Name, Class Name, Tag Name, and Link Text, as well as CSS selectors, wildcard selectors and child-node selectors. The purpose is to select appropriate and reliable locators, including for dynamic or challenging elements. □filecite□turn2file1□L3-L12□

4. Application Under Test

The application used by the supplied framework is the TutorialsNinja Demo e-commerce website.

Website: <https://tutorialsninja.com/demo/>

The framework automates selected e-commerce interactions. The supplied implementation contains tests for opening the site, performing a valid login using CSV test data, and searching for the product “iPhone” and validating that products are displayed.

5. Problem Statement

Web applications contain repeated user interactions that need to be validated whenever the application changes. Manually repeating browser actions such as opening a website, logging in, and searching for products can be inefficient and can introduce variation in execution.

The project addresses this problem by creating a reusable Python Selenium framework in which browser setup, page interactions, test data, logging, screenshots and test execution are organized into separate components. This reduces duplication and provides a structured basis for repeated automated testing.

6. Objectives

40. Automate selected web browser interactions using Selenium WebDriver.
41. Validate key functions of the TutorialsNinja Demo e-commerce website.
42. Create reusable test components using a Page Object Model structure.
43. Use pytest for test discovery and execution.
44. Read login test data from a CSV file.
45. Apply configuration management rather than hard-coding environment settings throughout the framework.
46. Capture screenshots when a test fails.
47. Maintain execution logs in a dedicated reports location.
48. Generate an HTML test report through pytest HTML.
49. Create a framework that can be extended with additional test scenarios.

7. Scope of the Project

The current implementation is scoped to browser automation for the TutorialsNinja Demo website. The supplied test modules cover website opening, valid login, and product search. The framework also provides supporting infrastructure for configuration, CSV data, logging, screenshots and HTML reporting.

The scope does not claim complete end-to-end coverage of the e-commerce application. Additional workflows such as checkout, payment, cross-browser execution, parallel execution or CI/CD are future enhancements unless separately implemented.

8. Technologies and Tools Used

Technology/Tool	Purpose / Role
Python	Programming language used for the automation framework.
Selenium WebDriver	Browser control and web-element interaction.
pytest	Test discovery, execution and assertions.
pytest-html	HTML test-report generation.
Page Object Model	Organization of page-specific browser interactions.
CSV	Storage of login test data.
configparser / INI configuration	Reading framework configuration.
Python logging	Execution logging to file and console.
Chrome WebDriver	Browser session used by the supplied unittest setup.
TutorialsNinja Demo	Application under test.

The requirements file specifies selenium==4.9.0, pytest==9.1.1, pytest-html==4.2.0, and pandas==3.0.5.

9. System Requirements

9.1 Hardware Requirements

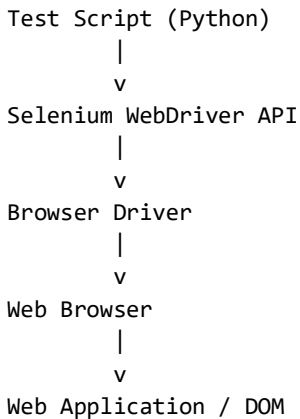
- A computer capable of running Python, a modern web browser, and Selenium WebDriver.
- Sufficient memory and storage for the Python environment, browser and generated reports/screenshots.

Exact minimum CPU, RAM and storage specifications are not specified in the supplied project materials.

9.2 Software Requirements

Component	Specified Information
Python	Python 3.14.2 was used in the project environment.
Selenium	4.49.0
PyTest	9.1.1
pytest-html	4.2.0
pandas	3.0.5
Browser	Chrome is used in the supplied unittest browser setup.
Editor/IDE	The supplied materials do not establish a mandatory IDE.
Virtual environment	venv is part of the project structure.

10. Selenium Architecture



The Python test invokes Selenium WebDriver methods. WebDriver communicates with the browser-specific driver, which controls the browser session. The browser loads the web application and exposes its page structure for Selenium to locate and interact with elements.

10.1 Communication Flow

50. The test initializes a WebDriver session.
51. The driver opens the configured application URL.
52. The test locates elements using Selenium locator strategies.
53. Commands such as click, send_keys and page navigation are executed through WebDriver.
54. Assertions validate the expected state.
55. The test framework closes the browser during teardown.

11. Project Framework Architecture

```

Selenium_Ecommerce_Framework/
├── .gitignore
├── conftest.py
├── pytest.ini
├── README.md
├── requirements.txt
├── base/

```

```

├── __init__.py
├── base_test.py
├── config/
│   └── config.ini
├── pages/
│   ├── __init__.py
│   ├── login_page.py
│   └── product_search_page.py
├── reports/
│   ├── automation.log
│   └── test_report.html
├── screenshots/
├── test_data/
│   └── login_data.csv
├── tests/
│   ├── __init__.py
│   ├── test_browser.py
│   ├── test_login.py
│   └── test_product_search.py
└── utilities/
    ├── __init__.py
    ├── config_reader.py
    ├── csv_reader.py
    ├── logger.py
    └── screenshot.py

```

Component	Responsibility
base/	Base test infrastructure, including shared browser setup.
pages/	Page Object classes for login and product search interactions.
tests/	PyTest test cases.
utilities/	Configuration, CSV, logging and screenshot utilities.
config/	Framework configuration file.
test_data/	Login test data stored as CSV.
reports/	Automation log and HTML report output.
screenshots/	Failure screenshots.
conftest.py	PyTest hook for automatic screenshots after failed call-phase tests.
pytest.ini	PyTest discovery and execution configuration.
requirements.txt	Pinned Python package dependencies.

12. Page Object Model

The Page Object Model separates page-specific interaction logic from test-case logic. In this project, LoginPage contains login-related operations and ProductSearchPage contains product-search-related operations.

This design allows tests to read like business-level scenarios while locator and interaction details remain inside page classes. It also reduces duplication when the same page operation is required by multiple tests.

13. Configuration Management

ConfigReader uses Python's configparser module and locates config/config.ini relative to the project root. This allows configuration values such as browser, base URL and implicit wait to be read from a centralized configuration file.

```
config = ConfigReader.get_config()
```

Centralized configuration makes the framework easier to maintain because environment-related values do not need to be repeated across test modules.

14. CSV Test Data

CSVReader reads login_data.csv from the test_data directory using Python's csv module and csv.DictReader. The valid-login test obtains the first record and reads the email and password fields.

```
test_data = CSVReader.read_login_data()
email = test_data[0]["email"]
password = test_data[0]["password"]
```

The project keeps the login-data file outside the source-code logic. The .gitignore also marks the credentials file under a test-credentials comment, which is a project cleanliness and security consideration.

15. PyTest Configuration and Test Discovery

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
addopts = -v
```

The configuration directs PyTest to the tests directory, recognizes files beginning with test_, classes beginning with Test, and functions beginning with test_. The -v option enables verbose test output.

16. Selenium Locators

The supplied course material requires practice with ID, Name, Class Name, Tag Name and Link Text, multiple-element identification, CSS selectors including wildcards, and CSS child selectors.

□filecite□turn2file1□L14-L30□

Locator	Definition	Generic Syntax	Project-specific evidence
ID	Finds an element by its id attribute.	By.ID, "value"	Specific page locator details are contained in page-object files; exact locator strings are not reproduced where not needed.
Name	Finds an element by name attribute.	By.NAME, "value"	Used as a supported Selenium locator concept in the course material.
Class Name	Finds by class attribute.	By.CLASS_NAME, "value"	Covered by the supplied course outcome.

Locator	Definition	Generic Syntax	Project-specific evidence
Tag Name	Finds elements by HTML tag.	By.TAG_NAME, "a"	Covered by the supplied course assignment.
Link Text	Finds an anchor by visible text.	By.LINK_TEXT, "text"	Covered by the supplied course assignment.
XPath	Uses a path expression to identify DOM nodes.	By.XPATH, "//..."	Exact project-specific XPath usage is not stated in the loaded course material.
CSS Selector	Uses CSS selector syntax.	By.CSS_SELECTOR, "..."	Course material explicitly includes CSS and child selectors.
Wildcard CSS	Matches varying attribute values.	[id^='user_']	The course material gives an ID-prefix wildcard example.
Child CSS	Targets nested children.	div > button	The course material gives a button-inside-div example.

17. Selenium WebDriver Commands

Command / Operation	Purpose	Project evidence / usage
driver.get(url)	Loads a web page.	Used in browser setup to open base_url.
driver.title	Reads the current page title.	Used by browser, login and product-search tests for output/validation.
driver.current_url	Reads the current URL.	Printed in login and product-search tests.
find_element(...)	Locates a web element.	Used through page objects and Selenium interactions.
click()	Clicks an element.	Relevant to page interactions; exact calls are encapsulated in page objects.
send_keys(...)	Types text into an input.	Used for login/search interactions through page objects.
clear()	Clears an input.	General WebDriver interaction; exact project usage is not established by the supplied test files.
implicitly_wait(...)	Sets an implicit wait.	Used in the supplied unittest browser setup.
quit()	Ends the browser session.	Used during teardown.

18. Explicit and Implicit Waits

Synchronization is important because web elements may not be immediately ready when a page starts loading. The supplied framework README identifies explicit waits and implicit waits as framework features.

18.1 Implicit Wait

An implicit wait tells WebDriver to poll for an element for a specified period before raising a not-found exception. The supplied unittest browser setup applies the configured value using `implicitly_wait(implicit_wait)`.

18.2 Explicit Wait

An explicit wait targets a specific condition and is normally used when a particular element or state needs to become available. The README identifies explicit waits as part of the framework, but the exact expected-condition implementation is not specified in the loaded test files.

18.3 Difference

Implicit waits apply generally to element lookup, while explicit waits are applied to specific conditions. They serve different synchronization purposes and should be used deliberately.

19. Assertions

Assertions compare an observed application state with an expected condition. In this project, assertions are used to validate that the website title contains “Your Store” and that login/search outcomes satisfy their expected conditions.

```
assert "Your Store" in self.driver.title
assert login_page.is_login_successful(), "Login was not successful"
assert product_search_page.are_products_displayed(), \
    "No products were displayed in search results"
```

A failed assertion marks the test as failed and can trigger the screenshot hook described later.

20. Browser Test

File: tests/test_browser.py

The TestBrowser class inherits from BaseTest and defines `test_open_website`. The test verifies that the browser opened the expected application by checking whether “Your Store” appears in the page title.

```
from base.base_test import BaseTest

class TestBrowser(BaseTest):
    def test_open_website(self):
        assert "Your Store" in self.driver.title
```

This is a basic availability/title validation and demonstrates the shared BaseTest browser setup.

21. Login Test

File: tests/test_login.py

The login test obtains credentials through CSVReader, creates a LoginPage object using the existing WebDriver, calls the page object's login method, prints the current URL and page title, and asserts that the page object reports a successful login.

```
from base.base_test import BaseTest
from pages.login_page import LoginPage
from utilities.csv_reader import CSVReader
```

```

class TestLogin(BaseTest):
    def test_valid_login(self):
        test_data = CSVReader.read_login_data()
        email = test_data[0]["email"]
        password = test_data[0]["password"]
        login_page = LoginPage(self.driver)
        login_page.login(email, password)
        print("Current URL:", self.driver.current_url)
        print("Page Title:", self.driver.title)
        assert login_page.is_login_successful(), \
            "Login was not successful"

```

The test deliberately keeps the credential-reading logic separate from the page interaction logic.

22. Product Search Test

File: tests/test_product_search.py

The product-search test creates a ProductSearchPage object, searches for “iPhone”, prints URL/title/search heading information, and verifies that products are displayed.

```

from base.base_test import BaseTest
from pages.product_search_page import ProductSearchPage

class TestProductSearch(BaseTest):
    def test_product_search(self):
        product_search_page = ProductSearchPage(self.driver)
        product_search_page.search_product("iPhone")
        print("Current URL:", self.driver.current_url)
        print("Page Title:", self.driver.title)
        print("Search Results Heading:",
              product_search_page.get_search_results_heading())
        assert product_search_page.are_products_displayed(), \
            "No products were displayed in search results"

```

This test demonstrates parameterized page interaction at the test level: the product term is passed to the page object's search method.

23. Automatic Failure Screenshots

The conftest.py file implements a pytest_runtest_makereport hookwrapper. It checks the call-phase result and, when a test fails, obtains the test instance and captures a screenshot while the browser is still open.

```

if report.when == "call" and report.failed:
    test_instance = getattr(item, "instance", None)
    if test_instance and hasattr(test_instance, "driver"):
        ScreenshotUtil.take_screenshot(
            test_instance.driver,
            test_name
        )

```

ScreenshotUtil creates the screenshots directory if necessary, adds a timestamp to the filename, calls driver.save_screenshot(), and returns the saved path. This provides visual evidence for failed tests.

24. Logging

`Logger.get_logger(name)` creates the reports directory and configures a file handler and console handler. The logger uses INFO level and the format “timestamp - level - logger name - message”. A duplicate-handler check prevents repeated handlers from being attached.

The primary file target is `reports/automation.log`. This creates a persistent execution trail that can support troubleshooting.

25. Test Cases

Test Case ID	Scenario	Preconditions	Steps	Expected Result	Actual Result / Status
TC-01	Open website	Browser setup is available.	Open configured base URL; read title.	Page title contains “Your Store”.	Not specified as a numeric result in the supplied materials.
TC-02	Valid login	Login data CSV is available.	Read first CSV record; open LoginPage; call login(email,password); verify success.	Login page object reports successful login.	Not specified as a numeric result in the supplied materials.
TC-03	Product search	Browser setup is available.	Create ProductSearchPage; search for “iPhone”; read heading; verify products displayed.	Search results contain displayed products.	Not specified as a numeric result in the supplied materials.

The three test cases above are extracted from the supplied PyTest test modules. The course assignment materials separately describe four learning assignments covering element identification, multiple elements, CSS selectors, and CSS child nodes. `filecite` `turn2file1` `L14-L30`

26. Implementation

26.1 Base Test and Browser Setup

BaseTest is the common parent class for the PyTest test modules. The supplied project structure places it in `base/base_test.py`. A separate unittest example demonstrates the underlying browser setup: configuration is read, Chrome is initialized with `--start-maximized`, the implicit wait is configured, and `base_url` is opened.

```
config = ConfigReader.get_config()
# browser/base_url/implicit_wait are read from configuration
options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)
driver.implicitly_wait(implicit_wait)
driver.get(base_url)
```

26.2 Page Objects

LoginPage and ProductSearchPage encapsulate page-level operations. The test modules therefore focus on scenario flow and assertions rather than directly containing all element interaction details.

26.3 Utilities

The utility layer contains ConfigReader, CSVReader, Logger and ScreenshotUtil. Each utility has one clear responsibility, making the framework easier to extend and maintain.

26.4 PyTest Hook

confest.py connects test failures with ScreenshotUtil. This is an example of framework-level behavior that applies automatically to test execution rather than requiring each test case to write screenshot code.

27. Code Explanation

27.1 Importing WebDriver

```
from selenium import webdriver
```

This imports Selenium's Python WebDriver interface, which is used to create and control a browser session.

27.2 Creating Chrome

```
driver = webdriver.Chrome(options=options)
```

This creates a Chrome WebDriver instance using the configured Chrome options.

27.3 Opening the Application

```
driver.get(base_url)
```

The get method navigates the browser to the configured application URL.

27.4 Reading the Title

```
assert "Your Store" in self.driver.title
```

driver.title returns the current page title. The assertion checks that the expected store title is present.

27.5 Reading CSV Data

```
test_data = CSVReader.read_login_data()
email = test_data[0]["email"]
password = test_data[0]["password"]
```

The CSV reader returns records as dictionaries. The first record's email and password values are passed to the LoginPage.

27.6 Searching for a Product

```
product_search_page.search_product("iPhone")
```

The test sends the product term to the page object. The page object is responsible for the actual Selenium element interactions.

27.7 Test Cleanup

```
driver.quit()
```

quit closes the WebDriver session and releases the browser resources.

28. Test Execution

PyTest discovers test files according to pytest.ini. The configured patterns are test_*.py for files, Test* for classes, and test_* for functions. Verbose output is enabled through addopts = -v.

pytest

The supplied project includes reports/test_report.html, indicating HTML reporting is part of the framework. However, the exact final aggregate count, pass/fail/skip totals and execution duration are not specified in the currently loaded source material and are therefore not fabricated in this report.

30. Test Report and HTML Reporting

pytest-html is included in requirements.txt and the project structure contains reports/test_report.html. An HTML test report can provide a human-readable summary of test execution, including individual test outcomes and execution details depending on the report configuration.

Because the actual HTML report contents are not reproduced in the supplied textual materials, this report does not claim a specific pass/fail count or execution duration. The report file's existence demonstrates that HTML reporting is part of the framework structure.

31. Screenshots

The project framework includes a screenshots directory and an automatic screenshot utility. The supplied textual project materials do not contain embedded browser screenshots that can be safely reproduced as figures in this document.

Figure 1: Failure Screenshot Mechanism — The framework captures a screenshot after a call-phase test failure while the browser is still available.

Figure 2: Screenshot File Naming — ScreenshotUtil generates a timestamped PNG filename under the screenshots directory.

Note: These are descriptions of implemented screenshot functionality, not fabricated screenshots.

32. Advantages

- Repeatable browser automation.
- Reusable Page Object Model structure.
- Separation of test logic, page logic and utilities.
- CSV-driven login test data.
- Centralized configuration.
- Automatic failure screenshots.
- File and console logging.
- PyTest discovery and verbose execution.
- HTML test-report support.
- Clear project directory organization.

33. Limitations

33.1 Project-Specific Limitations

The supplied source material does not provide a formal list of limitations observed during implementation. Therefore, no specific failure or limitation is attributed to the project without evidence.

33.2 General Selenium Limitations

- Dynamic web elements can require robust locator and synchronization strategies.
- CAPTCHA mechanisms are generally designed to resist browser automation.
- Browser and driver compatibility must be maintained.
- Network availability can affect web-based test execution.
- UI changes can invalidate page locators.
- Authentication flows may require environment-specific handling.

34. Challenges Faced

The supplied materials do not document a formal challenge log. The following are therefore presented as general engineering considerations rather than claims that they were encountered: locator stability, synchronization, browser-driver compatibility, dependency management, and maintaining test data separately from code.

The framework's use of configuration management, waits, Page Objects, logging and failure screenshots provides mechanisms that help address common automation-maintenance concerns.

35. Future Scope

The following are future enhancements, not claims about the current implementation:

- Expand the Page Object Model to cover additional e-commerce workflows.
- Increase data-driven coverage using additional test-data records.
- Add cross-browser test execution.
- Introduce parallel execution for larger suites.
- Integrate execution into CI/CD pipelines.
- Use Selenium Grid or cloud browser platforms for distributed testing.
- Add richer reporting and historical trend analysis.
- Containerize the automation environment with Docker.
- Strengthen test-data and secret-management practices.
- Add broader regression scenarios such as cart and checkout validation, subject to application requirements.

36. Conclusion

The Python Selenium Automation project demonstrates how browser automation can be organized into a reusable testing framework rather than a collection of isolated scripts. The supplied implementation combines Selenium WebDriver with PyTest, Page Object Model classes, configuration management, CSV test data, logging, automatic failure screenshots and HTML reporting.

The three supplied PyTest scenarios focus on opening the TutorialsNinja Demo site, validating a valid login flow, and searching for an iPhone product and confirming displayed results. The accompanying course

material reinforces the importance of web-element identification, reliable locators, multiple-element handling, CSS selectors, wildcard selectors and child-node selectors. □ filecite□turn2file1□L14-L30□

Overall, the project provides practical exposure to browser automation, test organization, assertions, reusable components and test evidence. Exact execution metrics should be read from the generated execution report when submitting measured results.

37. References

56. Project materials supplied for the Python Selenium Automation assignment, including the Selenium E-Commerce Automation Framework source files and course-outcome/assignment material.
57. Official Selenium documentation: Selenium WebDriver and Selenium documentation.
58. Official Python documentation: Python language and standard-library documentation.
59. Official PyTest documentation: test discovery, fixtures and test execution.
60. Official pytest-html documentation for HTML reporting.

URLs and detailed publication metadata are not reproduced here because the supplied project materials do not specify a bibliography. The report should use the institution's preferred citation style if a formal reference list is required.

Appendix A. Key Project Files

File	Role
conftest.py	PyTest hook for automatic screenshots on failed tests.
pytest.ini	PyTest discovery configuration.
requirements.txt	Pinned Python dependencies.
tests/test_browser.py	Website title test.
tests/test_login.py	Valid login test using CSV data and LoginPage.
tests/test_product_search.py	Product search test using ProductSearchPage.
utilities/config_reader.py	Reads config/config.ini.
utilities/csv_reader.py	Reads login_data.csv.
utilities/logger.py	Creates file and console logging.
utilities/screenshot.py	Creates timestamped failure screenshots.
pages/login_page.py	Encapsulates login interactions.
pages/product_search_page.py	Encapsulates product-search interactions.
base/base_test.py	Shared base test/browser infrastructure.
config/config.ini	Centralized framework configuration.
reports/automation.log	Execution log target.
reports/test_report.html	HTML report target.
screenshots/	Failure screenshot storage.
test_data/login_data.csv	Login test data.